

Project2 项目报告

姓名: 徐锦云

学号: 23307130447

Project2 项目报告

1. 任务一: 控制流图构建

1.1 实现思路

1.2 代码实现

- (1) 指令收集: 覆盖函数范围
- (2) 起始/终止识别: 规则化 + 特例兜底
- (3) 构造基本块: 按起点逐块闭包
- (4) 连边与标签
- (5) 可达性裁剪与跳板块合并

完整代码

1.3 结果分析

2. 任务二: 结构化代码生成

2.1 实现思路

2.2 代码实现

- (A) 调度与循环框架
- (B) 通用 goto 规则
- (C) 循环类规则
- (D) 顺序与分支类规则
- (E) 循环内控制语句规则

完整代码

额外的优化

2.3 结果分析

- 1) SEQUENCE: sequence_1052c
- 2) WHILE_TRUE: loop_while_true_10420
- 3) DO_WHILE: loop_while_10420
- 4) IF: branch_if_10420
- 5) IF_ELSE: branch_if_else_10420
- 6) IF_CONTINUE: loop_while_continue_10450
- 7) IF_BREAK: httpd_209AC
- 8) GOTO_SEQUENCE / GOTO_BRANCH: httpd_20814

总结

1. 任务一: 控制流图构建

1.1 实现思路

本任务的目标是把函数入口地址对应的机器码指令序列转换为满足“单入口、单出口”约束的基本块图结构。我的实现遵循“先全量反汇编，再切块，再连边，最后清理”的流程：

- 从入口地址出发进行线性反汇编，同时利用每条指令的后继地址维护函数可达地址上界，确保获取完整指令集。
- 基于前驱/后继关系识别基本块起始指令与终止指令，并对“无条件跳转到顺序下一条”的冗余跳转做特殊处理，避免产生伪块。

3. 以起始地址为锚点构造基本块，块内按地址顺序聚合指令，直到遇到终止点或下一个块起点。
4. 根据块尾指令后继关系连接 CFG 边；条件分支边保留 `T/F` 标签。
5. 设置入口块后，执行“从入口可达性过滤”，删除游离节点；最后对“单条无条件跳转桩块”进行安全合并，进一步提升图结构质量。

该思路兼顾了正确性（边关系与执行语义一致）与鲁棒性（对编译器产生的跳板块、线性扫描噪声有处理）。

1.2 代码实现

实现位于 `poredec/frontend/cfg_builder.py`，核心逻辑在 `build_cfg()`，另外新增了 `_merge_single_ins_uncond_jump_stubs()` 作为后处理。

(1) 指令收集：覆盖函数范围

在 `build_cfg()` 中，先通过 `curr_addr` 线性推进反汇编；循环条件采用“当前不是 `ret` 或者尚未达到已知最大后继地址”：

```
"""
步骤1：从入口地址开始反汇编，依次获取所有指令
下面代码演示了如何在给定地址(addr)处反汇编一条指令
>>> insn = self.lifter.disassemble_at(addr)
insn中包含了反汇编得到的指令信息，例如insn.succs表示该指令后继指令地址列表。更多信息见
class:`DisasInsn`
"""

insns: list[DisasInsn] = []
# TODO: 实现步骤1
# 获取函数所有指令
curr_addr = entry_addr
# 维护一个最大地址，以便判断是否到达函数末尾
max_addr = entry_addr
while self.lifter.disassemble_at(curr_addr).is_ret is False or curr_addr < max_addr:
    curr_insn = self.lifter.disassemble_at(curr_addr)
    for i in range(len(curr_insn.succs)):
        if curr_insn.succs[i] > max_addr:
            max_addr = curr_insn.succs[i]
    insns.append(curr_insn)
    curr_addr += curr_insn.size

# 获取ret指令
curr_insn = self.lifter.disassemble_at(curr_addr)
insns.append(curr_insn)
```

- 变量 `max_addr` 动态记录所有已见后继地址的最大值；
- 每次 `disassemble_at(curr_addr)` 后，把后继地址更新到 `max_addr`；
- 即使遇到一次 `ret`，只要后续还有可能是该函数内可达块，也会继续收集；
- 最后再补收当前 `ret` 指令。

这样避免了“过早遇到 `ret` 就提前结束”造成的块缺失问题。

(2) 起始/终止识别：规则化 + 特例兜底

"""

步骤2：识别基本块的起始指令和终止指令。

起始指令：有>1或0条前继指令，或前继指令中包含终止指令

终止指令：有>1或0条后继指令，或后继指令中包含起始指令

"""

TODO：实现步骤2

start_insn_addr: list[int] = []

end_insn_addr: list[int] = []

start_insn_addr.append(insns[0].address) # 起始指令地址列表初始化

end_insn_addr.append(curr_addr) # 终止指令地址列表初始化

先建 succ 边的前驱表，供下面判断「跳到下一条」是否可合并（无其它边汇入目标）

insn_addr_set = {insn.address for insn in insns}

preds_map: dict[int, set[int]] = {insn.address: set() for insn in insns}

for insn in insns:

for succ_addr in insn.succs:

if succ_addr in insn_addr_set:

preds_map[succ_addr].add(insn.address)

首先根据后继指令条数存入部分终止指令

for i in range(len(insns)):

if len(insns[i].succs) > 1 or len(insns[i].succs) == 0:

if insns[i].address not in end_insn_addr:

end_insn_addr.append(insns[i].address)

无条件跳转：仅当确实打断控制流时才视为终止。

若目标就是顺序下一条（如 b #下一行），且该地址只有本条指令作为 succ 来源，

则不应拆块，否则会产生单指令块 + 无意义后继入口（如 httpd 209AC）。

if len(insns[i].succs) == 1 and insns[i].is_cond is False and insns[i].is_jump

is True:

fallthrough_addr = insns[i].address + insns[i].size

target = insns[i].succs[0]

redundant_branch_to_next = target == fallthrough_addr and len(
 preds_map.get(target, ())

) == 1

if not redundant_branch_to_next:

if insns[i].address not in end_insn_addr:

end_insn_addr.append(insns[i].address)

兜底：若唯一后继不是顺序下一条指令，则也视为基本块终止。

if len(insns[i].succs) == 1:

fallthrough_addr = insns[i].address + insns[i].size

if insns[i].succs[0] != fallthrough_addr:

if insns[i].address not in end_insn_addr:

end_insn_addr.append(insns[i].address)

for addr, preds in preds_map.items():

if len(preds) != 1 and addr not in start_insn_addr:

start_insn_addr.append(addr)

终止指令的后继标记为起始指令

for i in range(len(insns)):

if insns[i].address in end_insn_addr:

```

        for j in range(len(insns[i].succs)):
            if insns[i].succs[j] not in start_insn_addr:
                start_insn_addr.append(insns[i].succs[j])

# 最后根据起始指令的前驱，找到所有的终止指令
for i in range(len(insns)):
    for j in range(len(insns[i].succs)):
        if insns[i].succs[j] in start_insn_addr and insns[i].address not in
end_insn_addr:

            end_insn_addr.append(insns[i].address)

# 将起始指令地址以及终止指令地址升序排列
start_insn_addr.sort()
end_insn_addr.sort()

```

我实现了三层识别逻辑：

- 先构造 `preds_map`（地址级前驱表），用于后续判断某个入口是否被多处汇入；
- 按“后继数为 0 或 >1”直接判定终止指令；
- 对“无条件跳转”做精细判断：若仅跳向顺序下一条且目标仅有当前指令一个前驱，则不强制切块；否则判为终止。

我在代码中处理了“冗余 branch-to-next”场景，避免把语义连续的线性段人为切开。之后再通过：

- “终止指令后继必为块起点”
- “若某指令跳到某个起点，则该指令为终止”

两条规则补全 `start_insn_addr / end_insn_addr` 集合，最后排序，保证切块流程可重复、确定。

(3) 构造基本块：按起点逐块闭包

```

"""
步骤3：构造基本块，基本块包含从起始指令到下一条终止指令间的所有指令。
下面代码演示了如何创建一个基本块，node_insns为基本块中的所有指令
>>> node_addr = node_insns[0].address # 获取基本块的起始地址
>>> node = CFGNode(node_addr, node_insns) # 创建基本块
创建的基本块可以通过下面代码添加到控制流图中
>>> cfg.add_node(node)
"""

# TODO: 实现步骤3
nodes_total: list[CFGNode] = []
node_by_start_addr: dict[int, CFGNode] = {}
insn_by_addr = {insn.address: insn for insn in insns}
start_insn_addr_set = set(start_insn_addr)
end_insn_addr_set = set(end_insn_addr)

# 每个基本块从start开始，直到遇到end为止；若中途遇到下一个start则结束当前块。
for i, node_start_addr in enumerate(start_insn_addr):
    node_insns: list[DisasInsn] = []
    curr_addr = node_start_addr
    next_start_addr = (
        start_insn_addr[i + 1] if i + 1 < len(start_insn_addr) else None

```

```

    )
    while curr_addr in insn_by_addr:
        node_insns.append(insn_by_addr[curr_addr])
        if curr_addr in end_insn_addr_set:
            break

        curr_insn = insn_by_addr[curr_addr]
        next_addr = curr_addr + curr_insn.size
        if (
            next_start_addr is not None
            and next_addr >= next_start_addr
            and curr_addr not in end_insn_addr_set
        ):
            break
        if next_addr in start_insn_addr_set and next_addr != node_start_addr:
            break
        curr_addr = next_addr

    node = CFGNode(node_start_addr, node_insns)
    cfg.add_node(node)
    nodes_total.append(node)
    node_by_start_addr[node_start_addr] = node

```

我使用了 `node_by_start_addr` 与 `insn_by_addr` 两个索引，加速“地址 -> 指令/节点”映射。每个块按以下终止条件闭包：

- 命中终止指令地址；
- 到达下一个块起始地址边界；
- 下一条指令本身就是另一个块起点。

然后创建 `CFGNode(node_start_addr, node_insns)` 并入图。该实现确保“同一地址只属于一个块”，满足实验要求。

(4) 连边与标签

```

"""
步骤4：根据控制流连接基本块
下面代码演示了如何添加控制流图的边
>>> cfg.add_edge(u, v) # 添加一条从u到v的边
>>> cfg.add_edge(u, v, label="T") # 可以指定边的标签，标签会在画图时显示
"""

# TODO: 实现步骤4
for i, node in enumerate(nodes_total):
    if not node.insns:
        continue
    node_end_insn = node.insns[-1]
    for j in range(len(node_end_insn.succs)):
        # 如果某个起始指令是该终止指令的后继指令，则添加一条从终止到起始指令的边
        if node_end_insn.succs[j] in start_insn_addr:
            succ_node = node_by_start_addr[node_end_insn.succs[j]]

            # 终止语句是条件跳转语句，需要指定边的标签

```

```

        if len(node_end_insn.succs) == 2:
            if j == 0:
                cfg.add_edge(node, succ_node, label="T")
            else:
                cfg.add_edge(node, succ_node, label="F")
        # 其他情况无需加入标签
    else:
        cfg.add_edge(node, succ_node)

```

我以“块尾指令”作为出边来源，遍历其 `succs`：

- 单后继/无条件边：直接 `cfg.add_edge(node, succ_node)`；
- 双后继/条件边：按后继顺序附加 `label="T"` 与 `label="F"`。

这使得后续结构化阶段可以直接利用边标签与分支顺序恢复条件语义。

(5) 可达性裁剪与跳板块合并

```

"""
步骤5：设置控制流图的入口节点
"""

# TODO：实现步骤5
cfg.set_entry(nodes_total[0])

# 仅保留从入口可达的节点，过滤线性反汇编扫描产生的游离块。
reachable_nodes = set()
queue: Queue[CFGNode] = Queue()
queue.put(cfg.entry)
reachable_nodes.add(cfg.entry)
while not queue.empty():
    curr_node = queue.get()
    for succ in cfg.successors(curr_node):
        if succ in reachable_nodes:
            continue
        reachable_nodes.add(succ)
        queue.put(succ)
for node in list(cfg.nodes):
    if node not in reachable_nodes:
        cfg.remove_node(node)

_merge_single_ins_uncond_jump_stubs(cfg)

assert cfg.nodes, "控制流图中没有节点"
assert cfg.entry, "需要设置控制流图的入口节点"
return cfg

```

在 `cfg.set_entry(nodes_total[0])` 后，我新增了两个后处理：

1. **入口可达性过滤**：从入口做 BFS，仅保留可达节点，删除线性扫描带来的游离块；
2. **单指令无条件跳转桩块合并**：`_merge_single_ins_uncond_jump_stubs()` 在满足“桩块只有一个后继、后继只有该桩块一个前驱”等安全条件时，将 `u -> v` 合并为新节点 `w`，并保留入边/出边。

我还考虑了 `CFGNode.__hash__` 依赖地址的问题，采用“删旧建新节点”而非原地改地址，避免破坏 NetworkX 图内部一致性。

完整代码

```
"""
控制流图构建器
"""

# pylint: disable=pointless-string-statement
# pylint: disable=unused-argument
# pylint: disable=fixme

from ..models.insn import DisasInsn
from ..models.cfg import CFGNode, ControlFlowGraph
from .lifter import Lifter
from queue import Queue

__all__ = ["CFGBuilder"]

def _merge_single_ins_uncond_jump_stubs(cfg: ControlFlowGraph) -> None:
    """合并「单条无条件跳转 → 唯一前驱可达的后继块」的桩基本块。

    例如 ``b #0x20a4c`` 与顺序下一条之间隔着其它地址时，不会在指令层与后继线性相邻，
    但仍可安全合并当且仅当目标块在 CFG 中只有该跳转块作为前驱（无其它汇入边）。

    合并后新建 :class:`CFGNode`，不原地修改已入图节点的 ``address``，因为 ``CFGNode.__hash__``
    依赖 ``address``，会损坏 NetworkX 内部结构。
    """
    changed = True
    while changed:
        changed = False
        for u in list(cfg.nodes):
            succs = list(cfg.successors(u))
            if len(succs) != 1:
                continue
            v = succs[0]
            if u is v:
                continue
            if len(u.insns) != 1:
                continue
            br = u.insns[0]
            if not (br.is_jump and not br.is_cond and len(br.succs) == 1):
                continue
            if not v.insns or br.succs[0] != v.insns[0].address:
                continue
            preds_v = list(cfg.predecessors(v))
            if len(preds_v) != 1 or preds_v[0] is not u:
                continue

            stub_addr = u.address
            merged = list(u.insns) + list(v.insns)
```

```

        in_u = [(p, dict(cfg.get_edge_data(p, u) or {})) for p in
list(cfg.predecessors(u))]
        out_v = [(s, dict(cfg.get_edge_data(v, s) or {})) for s in
list(cfg.successors(v))]
        was_entry = cfg.entry is u or cfg.entry is v

        cfg.remove_node(u)
        cfg.remove_node(v)
        w = CFGNode(stub_addr, merged)
        cfg.add_node(w)
        for p, ed in in_u:
            if p is u or p is v:
                continue
            cfg.add_edge(p, w, **ed)
        for s, ed in out_v:
            if s is u or s is v:
                continue
            cfg.add_edge(w, s, **ed)
        if was_entry:
            cfg.set_entry(w)
        changed = True
        break

```

```

class CFGBuilder:

```

```

    """控制流图构建器"""

```

```

    def __init__(self, lifter: Lifter):
        self.lifter = lifter # 反汇编器

```

```

    def build_cfg(self, entry_addr: int) -> ControlFlowGraph:

```

```

        """从给定的函数起始地址开始，构建完整的函数控制流图。

```

```

        控制流图的节点为基本块class:`CFGNode`，边表示控制流转移。

```

```

        每个节点包含一到多条汇编指令class:`DisasInsn`。

```

```

        :param entry_addr: 函数的起始地址

```

```

        :return: 构建完成的控制流图

```

```

        """

```

```

        # 创建一个空的控制流图

```

```

        cfg = ControlFlowGraph()

```

```

        """

```

```

        步骤1：从入口地址开始反汇编，依次获取所有指令

```

```

        下面代码演示了如何在给定地址(addr)处反汇编一条指令

```

```

        >>> insn = self.lifter.disassemble_at(addr)

```

```

        insn中包含了反汇编得到的指令信息，例如insn.succs表示该指令后继指令地址列表。更多信息见

```

```

class:`DisasInsn`

```

```

        """

```

```

        insns: list[DisasInsn] = []

```

```

        # TODO: 实现步骤1

```

```

        # 获取函数所有指令

```

```

        curr_addr = entry_addr

```

```

        # 维护一个最大地址，以便判断是否到达函数末尾

```

```

        max_addr = entry_addr

```



```

while self.lifter.disassemble_at(curr_addr).is_ret is False or curr_addr < max_addr:
    curr_insn = self.lifter.disassemble_at(curr_addr)
    for i in range(len(curr_insn.succs)):
        if curr_insn.succs[i] > max_addr:
            max_addr = curr_insn.succs[i]
    insns.append(curr_insn)
    curr_addr += curr_insn.size

```

获取ret指令

```

curr_insn = self.lifter.disassemble_at(curr_addr)
insns.append(curr_insn)

```

"""

步骤2：识别基本块的起始指令和终止指令。

起始指令：有>1或0条前继指令，或前继指令中包含终止指令

终止指令：有>1或0条后继指令，或后继指令中包含起始指令

"""

TODO：实现步骤2

```

start_insn_addr: list[int] = []
end_insn_addr: list[int] = []
start_insn_addr.append(insns[0].address) # 起始指令地址列表初始化
end_insn_addr.append(curr_addr) # 终止指令地址列表初始化

```

先建 succ 边的前驱表，供下面判断「跳到下一条」是否可合并（无其它边汇入目标）

```

insn_addr_set = {insn.address for insn in insns}
preds_map: dict[int, set[int]] = {insn.address: set() for insn in insns}
for insn in insns:
    for succ_addr in insn.succs:
        if succ_addr in insn_addr_set:
            preds_map[succ_addr].add(insn.address)

```

首先根据后继指令条数存入部分终止指令

```

for i in range(len(insns)):
    if len(insns[i].succs) > 1 or len(insns[i].succs) == 0:
        if insns[i].address not in end_insn_addr:
            end_insn_addr.append(insns[i].address)

```

无条件跳转：仅当确实打断控制流时才视为终止。

若目标就是顺序下一条（如 b #下一行），且该地址只有本条指令作为 succ 来源，

则不应拆块，否则会产生单指令块 + 无意义后继入口（如 httpd 209AC）。

```

if len(insns[i].succs) == 1 and insns[i].is_cond is False and insns[i].is_jump

```

is True:

```

    fallthrough_addr = insns[i].address + insns[i].size
    target = insns[i].succs[0]
    redundant_branch_to_next = target == fallthrough_addr and len(
        preds_map.get(target, ())
    ) == 1
    if not redundant_branch_to_next:
        if insns[i].address not in end_insn_addr:
            end_insn_addr.append(insns[i].address)

```

兜底：若唯一后继不是顺序下一条指令，则也视为基本块终止。

```

if len(insns[i].succs) == 1:
    fallthrough_addr = insns[i].address + insns[i].size

```

```

        if insns[i].succs[0] != fallthrough_addr:
            if insns[i].address not in end_insn_addr:
                end_insn_addr.append(insns[i].address)
    for addr, preds in preds_map.items():
        if len(preds) != 1 and addr not in start_insn_addr:
            start_insn_addr.append(addr)

    # 终止指令的后继标记为起始指令
    for i in range(len(insns)):
        if insns[i].address in end_insn_addr:
            for j in range(len(insns[i].succs)):
                if insns[i].succs[j] not in start_insn_addr:
                    start_insn_addr.append(insns[i].succs[j])

    # 最后根据起始指令的前驱，找到所有的终止指令
    for i in range(len(insns)):
        for j in range(len(insns[i].succs)):
            if insns[i].succs[j] in start_insn_addr and insns[i].address not in
end_insn_addr:
                end_insn_addr.append(insns[i].address)

    # 将起始指令地址以及终止指令地址升序排列
    start_insn_addr.sort()
    end_insn_addr.sort()

    """

    步骤3：构造基本块，基本块包含从起始指令到下一条终止指令间的所有指令。
    下面代码演示了如何创建一个基本块，node_insns为基本块中的所有指令
    >>> node_addr = node_insns[0].address # 获取基本块的起始地址
    >>> node = CFGNode(node_addr, node_insns) # 创建基本块
    创建的基本块可以通过下面代码添加到控制流图中
    >>> cfg.add_node(node)
    """

    # TODO: 实现步骤3
    nodes_total: list[CFGNode] = []
    node_by_start_addr: dict[int, CFGNode] = {}
    insn_by_addr = {insn.address: insn for insn in insns}
    start_insn_addr_set = set(start_insn_addr)
    end_insn_addr_set = set(end_insn_addr)

    # 每个基本块从start开始，直到遇到end为止；若中途遇到下一个start则结束当前块。
    for i, node_start_addr in enumerate(start_insn_addr):
        node_insns: list[DisasInsn] = []
        curr_addr = node_start_addr
        next_start_addr = (
            start_insn_addr[i + 1] if i + 1 < len(start_insn_addr) else None
        )
        while curr_addr in insn_by_addr:
            node_insns.append(insn_by_addr[curr_addr])
            if curr_addr in end_insn_addr_set:
                break

            curr_insn = insn_by_addr[curr_addr]

```

```

        next_addr = curr_addr + curr_insn.size
        if (
            next_start_addr is not None
            and next_addr >= next_start_addr
            and curr_addr not in end_insn_addr_set
        ):
            break
        if next_addr in start_insn_addr_set and next_addr != node_start_addr:
            break
        curr_addr = next_addr

    node = CFGNode(node_start_addr, node_insns)
    cfg.add_node(node)
    nodes_total.append(node)
    node_by_start_addr[node_start_addr] = node

"""
步骤4：根据控制流连接基本块
下面代码演示了如何添加控制流图的边
>>> cfg.add_edge(u, v) # 添加一条从u到v的边
>>> cfg.add_edge(u, v, label="T") # 可以指定边的标签，标签会在画图时显示
"""

# TODO：实现步骤4
for i, node in enumerate(nodes_total):
    if not node.insns:
        continue
    node_end_insn = node.insns[-1]
    for j in range(len(node_end_insn.succs)):
        # 如果某个起始指令是该终止指令的后继指令，则添加一条从终止到起始指令的边
        if node_end_insn.succs[j] in start_insn_addr:
            succ_node = node_by_start_addr[node_end_insn.succs[j]]

            # 终止语句是条件跳转语句，需要指定边的标签
            if len(node_end_insn.succs) == 2:
                if j == 0:
                    cfg.add_edge(node, succ_node, label="T")
                else:
                    cfg.add_edge(node, succ_node, label="F")
            # 其他情况无需加入标签
            else:
                cfg.add_edge(node, succ_node)

"""
步骤5：设置控制流图的入口节点
"""

# TODO：实现步骤5
cfg.set_entry(nodes_total[0])

# 仅保留从入口可达的节点，过滤线性反汇编扫描产生的游离块。
reachable_nodes = set()
queue: Queue[CFGNode] = Queue()
queue.put(cfg.entry)
reachable_nodes.add(cfg.entry)

```

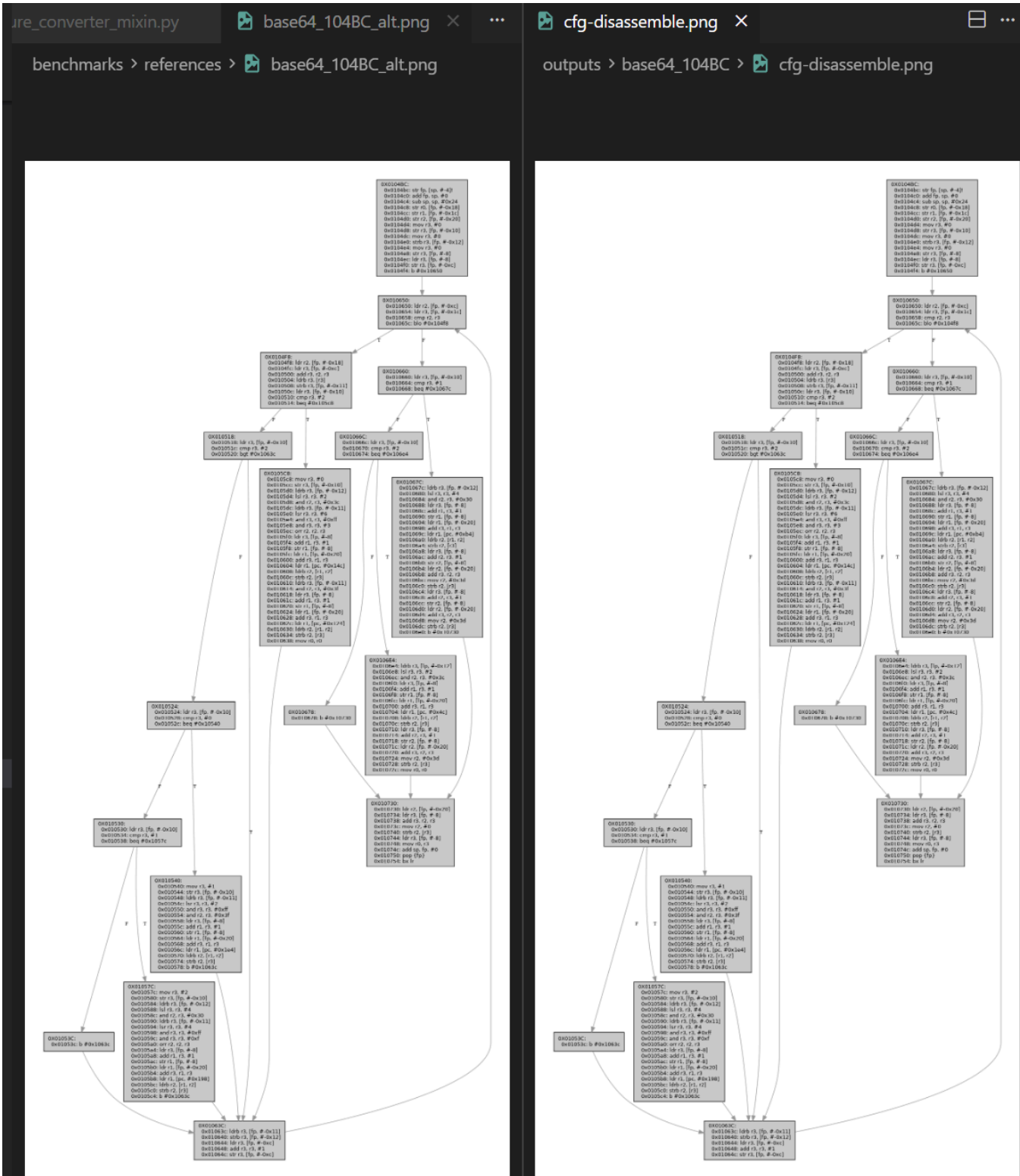
```
while not queue.empty():
    curr_node = queue.get()
    for succ in cfg.successors(curr_node):
        if succ in reachable_nodes:
            continue
        reachable_nodes.add(succ)
        queue.put(succ)
for node in list(cfg.nodes):
    if node not in reachable_nodes:
        cfg.remove_node(node)

_merge_single_ins_uncond_jump_stubs(cfg)

assert cfg.nodes, "控制流图中没有节点"
assert cfg.entry, "需要设置控制流图的入口节点"
return cfg
```

1.3 结果分析

我选取 `base64_104BC` 作为任务一分析样例。



1. 在反汇编阶段，目标函数全部指令被完整纳入分析范围，未出现块缺失。
2. 在切块阶段，冗余跳转没有被错误切分为额外入口块，避免图中出现不必要的单指令碎片节点。
3. 在连边阶段，条件边 **T/F** 标注与控制转移方向一致，为后续结构化还原提供了稳定输入。
4. 在后处理阶段，可达性裁剪去除了不可达噪声；跳板块合并减少了“技术性节点”，使最终 CFG 更贴近高级语言控制结构。

2. 任务二: 结构化代码生成

2.1 实现思路

结构化代码生成采用“优先规则化简 + 必要时 goto 兜底 + 循环专项处理”的三阶段策略:

1. 对每个节点优先尝试针对性规则 (sequence/if/if-else/do-while/while/if-continue/if-break) ;
2. 对无法规则化的结构, 使用 goto 规则 (顺序与分支) 保证 CFG 可持续收缩;
3. 若节点是循环头, 先在循环内部反复应用规则; 若仍未收敛, 则对循环体启用 goto 化简, 再回收成 while(1)。

整体目标是: 在保证可完全化简的前提下, 尽量减少残留 goto, 输出更结构化、更可读的伪 C 代码。

2.2 代码实现

实现位于 poredec/backend/structure_converter.py。

(A) 调度与循环框架

- reduce_cfg(): 统一封装 BFS 化简入口, 按 by_rule 选择规则化简或 goto 化简。

```
def reduce_cfg(self, node, by_rule, loop=None) -> bool:
```

```
    """递归化简控制流图, 尝试将多个节点化简为一个节点。
```

函数会使用广度优先搜索遍历控制流图中从给定节点可达的所有节点, 并在尝试化简这些节点——将节点与其后继节点合并。

```
    :param node: 待化简的节点
```

```
    :param by_rule: 为True时使用针对性规则化简, 否则使用通用(goto)规则
```

```
    :param loop: 当给定loop时, 只化简循环内的节点, 否则化简所有node可达的节点。
```

```
    :return: 如果节点被化简为结构化语句, 则返回True
```

```
    """
```

```
    if by_rule:
```

```
        reduce_node_func = self.reduce_node_by_rule
```

```
    else:
```

```
        reduce_node_func = self.reduce_node_by_goto
```

```
    if loop:
```

```
        return self.bfs_traverse_loop(node, reduce_node_func, loop)
```

```
    else:
```

```
        return self.bfs_traverse(node, reduce_node_func)
```

- reduce_node_by_rule(): 将规则触发顺序调整为

```
SEQUENCE -> WHILE_TRUE -> DO_WHILE -> IF_ELSE -> IF_CONTINUE -> IF_BREAK -> IF。
```

该顺序优先保留循环内部语义 (continue/break) 与双分支结构, 再降级到单分支 if, 可减少过早合并造成的信息损失。

```
def reduce_node_by_rule(self, node: CFGNode) -> bool:
```

```
    """检查给定的控制流图节点是否满足化简条件, 如果可以则将其化简为结构化语句。
```

函数的实现逻辑是：首先检查节点是否为循环头，如果是则尝试化简循环；否则尝试使用其他规则化简节点。

每个规则化简函数名以`try\_reduce\_`开头，例如`try\_reduce\_sequence`。

每个规则化简函数需要完成两方面操作：

1. 判断节点是否满足化简条件，如果不满足则返回False。

2. 如果满足则将节点化简为结构化语句，并返回True。

当任意节点被化简时，函数会调用`emit\_reduce\_event`方法记录化简事件，并返回True。

```
:param node: 待检查的控制流图节点
:return: 如果节点被化简为结构化语句，则返回True，否则返回False
"""
if self.is_loop_header(node) and not self.is_loop_header_under_analysis(node):
    self.enter_loop(node)
    self.completely_reduce_loop(node)
    self.leave_loop()
    return True

if self.try_reduce_sequence(node):
    self.emit_reduce_event(ReduceRule.SEQUENCE, node)
    return True

if self.try_reduce_while_true(node):
    self.emit_reduce_event(ReduceRule.WHILE_TRUE, node)
    return True

if self.try_reduce_do_while(node):
    self.emit_reduce_event(ReduceRule.DO_WHILE, node)
    return True

if self.try_reduce_if_else(node):
    self.emit_reduce_event(ReduceRule.IF_ELSE, node)
    return True

if self.try_reduce_if_continue(node, True) or
self.try_reduce_if_continue(node, False):
    self.emit_reduce_event(ReduceRule.IF_CONTINUE, node)
    return True

if self.try_reduce_if_break(node, True) or self.try_reduce_if_break(node,
False):
    self.emit_reduce_event(ReduceRule.IF_BREAK, node)
    return True

if self.try_reduce_if(node, True) or self.try_reduce_if(node, False):
    self.emit_reduce_event(ReduceRule.IF, node)
    return True

return False
```

- `completely_reduce_loop()`：循环头进入分析栈后，先尽可能规则化；若未化简完成，再调用 `by_rule=False` 的 `goto` 化简并补自环，最终尝试 `while(1)` 回收。这一框架确保了“尽量结构化、一定可收敛”。

```
def completely_reduce_loop(self, node: CFGNode):
```

```
    """完全化简以节点为循环头的循环，将循环内的节点化简为结构化语句。
```

该方法首先尝试使用针对性规则化简循环；如果无法化简则使用**goto**规则将循环内的后继节点与节点合并，

直到循环内只剩一个节点，从而可以将该节点转化为**while(1)**循环。

注意：如果循环没有被化简，大概率是通用(**goto**)规则实现不完整。

```
    :param node: 循环头部节点
```

```
    """
```

```
    loop = self.get_loop_for(node)
```

```
    while not self.is_loop_reduced(loop) and self.reduce_cfg(node, by_rule=True):
        pass
```

```
    if not self.is_loop_reduced(loop):
        self.reduce_cfg(node, by_rule=False, loop=loop)
        self.cfg.add_edge(node, node)
        if self.try_reduce_while_true(node):
            self.emit_reduce_event(ReduceRule.WHILE_TRUE, node)
```

(B) 通用 goto 规则

1. try_reduce_goto_sequence()

```
def try_reduce_goto_sequence(self, node: CFGNode) -> bool:
```

```
    """使用goto规则化简顺序结构。对于给定的节点，如果只有一个后继节点，则在给定节点后面添加goto语句，并删除后继节点。
```

具体来说::

识别如下结构

```
node {stmt1}
└─ succ {stmt2}
```

将其化简为

```
node {stmt1; goto stmt2;}
```

```
    :param node: 待化简的控制流图节点
```

```
    :return: 如果节点被化简为结构化语句，则返回True，否则返回False
```

```
    """
```

```
    succs: list[CFGNode] = self.successors(node)
```

```
    # 若后继数量不为1，则不符合化简条件
```

```
    if len(succs) != 1:
        return False
```

```
    goto_node = succs[0]
```

```
    # 排除自循环的情况
```

```
    if goto_node == node:
        return False
```

```
    # 若后继节点仅由当前节点到达，则优先直接拼接语句，减少残留goto目标块。
```

```
    preds = self.predecessors(goto_node)
```

```
    if len(preds) == 1 and preds[0] == node:
```



```

        self.reduce_node(node, goto_node.insns, self.successors(goto_node), True)
        return True

goto_stmt = self.gen_goto_stmt(goto_node)
self.reduce_node(node, [goto_stmt], new_succs=[], extend=True)
return True

```

- 条件: `node` 恰有 1 个非自环后继;
- 优化点: 若后继只有当前唯一前驱, 则不生成 `goto`, 而是直接拼接后继语句并继承其后继;
- 否则生成 `goto` 语句并收缩节点。

这条优化显著降低了纯跳转残留, 属于“语义等价下优先结构化”的正确做法。

2. `try_reduce_goto_branch()`

```

def try_reduce_goto_branch(self, node: CFGNode) -> bool:
    """使用goto规则化简if-else结构。对于给定的节点, 如果有两个后继节点, 则在节点后面添加if-
    goto-else-goto语句, 并删除后继节点。

    具体来说::

        识别如下结构
        node {stmt1}
        |— succ1 {stmt2}
        |— succ2 {stmt3}
        将其化简为
        node {stmt1; if(cond) goto stmt2; else goto stmt3;}

    :param node: 待化简的控制流图节点
    :return: 如果节点被化简为结构化语句, 则返回True, 否则返回False
    """
    succs: list[CFGNode] = self.successors(node)
    # 若后继数量不为2, 则不符合化简条件
    if len(succs) != 2:
        return False

    cond = self.get_cond_expr(node)
    succs1_node = succs[0]
    succs2_node = succs[1]
    goto_stmt1 = self.gen_goto_stmt(succs1_node)
    goto_stmt2 = self.gen_goto_stmt(succs2_node)
    if_stmt = self.gen_if_stmt(cond, [goto_stmt1], [goto_stmt2])
    self.reduce_node(node, [if_stmt], new_succs=[], extend=True)
    return True

```

- 条件: `node` 恰有 2 个后继;
- 动作: 生成 `if(cond) goto succ1; else goto succ2;`, 并通过 `reduce_node()` 收缩。

(C) 循环类规则

1. `try_reduce_while_true()`

```
def try_reduce_while_true(self, node: CFGNode) -> bool:
    """尝试将给定的节点化简为while(1)语句，避免使用goto

    具体来说::

        识别如下自循环结构
        node {stmt}
        └─ node (自循环)
        将其化简为
        node {while(1) {stmt}}

    :param node: 待化简的控制流图节点
    :return: 如果节点被化简为结构化语句，则返回True，否则返回False
    """
    succs: list[CFGNode] = self.successors(node)
    # 判断是否为自循环
    if len(succs) != 1 or succs[0] != node:
        return False

    while_stmt = self.gen_while_stmt(ConstExpr(1), node.insns)
    self.reduce_node(node, [while_stmt], new_succs=[], extend=False)
    return True
```

- 检测单自环结构 `succs == [node]`;
- 生成 `while(1)` 并以 `extend=False` 替换原语句体，防止重复累积。

2. `try_reduce_do_while()`

```
def try_reduce_do_while(self, node: CFGNode) -> bool:
    """尝试将给定的节点化简为do-while语句，避免使用goto

    具体来说::

        识别如下结构
        node {stmt}
        └─ node (自循环)
        └─ succ (循环后继节点)
        将其化简为
        node {do {stmt} while (1)}
        └─ succ

    :param node: 待化简的控制流图节点
    :return: 如果节点被化简为结构化语句，则返回True，否则返回False
    """
    succs: list[CFGNode] = self.successors(node)
    # 判断是否为do_while循环
    if len(succs) != 2:
        return False
```

```

if node not in succs:
    return False

# 确定循环后继节点
loop_succ = succs[1] if succs[0] == node else succs[0]
cond = self.get_cond_expr(node)
# get_cond_expr(node) 表示 True 分支 (succs[0]) 条件,
# 若自循环在 False 分支上, 需要取反。
if succs[1] == node:
    cond = cond.logical_not()
do_while_stmt = self.gen_do_while_stmt(cond, node.insns)
self.reduce_node(node, [do_while_stmt], new_succs=[loop_succ], extend=False)
return True

```

- 检测“双后继且包含自环”;
- 依据自环位于真分支还是假分支, 选择条件是否取反;
- 生成 `do {...} while(cond)`, 并保留循环后继 `loop_succ`。

该实现正确处理了 `get_cond_expr(node)` 与后继顺序间的映射关系, 是 do-while 还原正确性的关键。

(D) 顺序与分支类规则

1. `try_reduce_sequence()`

```

def try_reduce_sequence(self, node: CFGNode) -> bool:
    """尝试将简单顺序结构化简为单个节点, 避免使用goto

    具体来说::

        识别如下结构
        node {stmt1}
        └─ succ {stmt2} (succ != node, 即不是自循环)
        ...
        └─ succ_succ_N (N >= 0)
        将其化简为
        node {stmt1; stmt2}
        ...
        └─ succ_succ_N (N >= 0)

    :param node: 待化简的控制流图节点
    :return: 如果节点被化简为结构化语句, 则返回True, 否则返回False
    """
    succs: list[CFGNode] = self.successors(node)
    # 判断是否为顺序结构
    if len(succs) != 1:
        return False

    succ_node = succs[0]
    # 判断是否为自循环
    if succ_node == node:
        return False

```

```

# 判断是否有其他前驱节点
preds = self.predecessors(succ_node)
if len(preds) != 1 or preds[0] != node:
    return False

self.reduce_node(node, succ_node.insns, self.successors(succ_node), True)
return True

```

- 要求 `node -> succ` 且 `succ` 只有 `node` 一个前驱;
- 直接拼接 `succ.insns` 并接管其后继, 避免引入多余转移语句。

2. `try_reduce_if(node, true_branch)`

```

def try_reduce_if(self, node: CFGNode, true_branch: bool) -> bool:
    """尝试将给定的节点化简为if结构, 避免使用goto。
    当branch为True时, 将node的第一个后继视为if_node, 否则将另一个后继视为if_node

```

具体来说::

识别如下结构:

```

node {stmt}
├─ if_node {stmt1}
│   └─ if_succ
└─ if_succ

```

将其化简为

```

node {stmt; if(cond) {stmt1}}
└─ if_succ

```

```

:param node: 待化简的控制流图节点
:param true_branch: 尝试化简为if结构的分支方向, 为True时if语句的条件为`cond =
get_cond_expr(node)`,
    否则条件为`cond = cond.logical_not()`
:return: 如果节点被化简为结构化语句, 则返回True, 否则返回False
"""

succs: list[CFGNode] = self.successors(node)
if len(succs) != 2:
    return False

if_node = succs[0] if true_branch else succs[1]
if_succ = succs[1] if true_branch else succs[0]
if if_node == node or if_succ == node:
    return False

# 判断if_node的后继节点是否为if_succ
succs_if_node = self.successors(if_node)
if succs_if_node != [if_succ]:
    return False

cond = self.get_cond_expr(node)
if not true_branch:
    cond = cond.logical_not()

if_stmt = self.gen_if_stmt(cond, if_node.insns, None)

```

```

self.reduce_node(node, [if_stmt], [if_succ], True)
return True

```

- 通过参数控制将哪个后继视作 `if_node`;
- 要求 `if_node` 单后继且恰好汇入 `if_succ`;
- 根据分支方向对条件取反, 生成无 `else` 的 `if`。

3. `try_reduce_if_else()`

```

def try_reduce_if_else(self, node: CFGNode) -> bool:
    """尝试将给定的节点化简为if-else结构, 避免使用goto

```

具体来说::

识别如下CFG结构

```

node {stmt}
├─ if_node {stmt1}
│   └─ if_succ
└─ else_node {stmt2}
    └─ else_succ (if_succ == else_succ)

```

将其化简为

```

node {stmt; if(cond) {stmt1} else {stmt2};}
└─ if_succ

```

:param node: 待化简的控制流图节点

:return: 如果节点被化简为结构化语句, 则返回True, 否则返回False

"""

```

succs = self.successors(node)

```

```

if len(succs) != 2:
    return False

```

```

if_node = succs[0]

```

```

else_node = succs[1]

```

```

if if_node == node or else_node == node:
    return False

```

```

if if_node == else_node:
    return False

```

```

if_succs = self.successors(if_node)

```

```

if len(if_succs) != 1:
    return False

```

```

else_succs = self.successors(else_node)

```

```

if len(else_succs) != 1:
    return False

```

```

if_succ = if_succs[0]

```

```

else_succ = else_succs[0]

```

```

if if_succ != else_succ:
    return False

```

```

cond = self.get_cond_expr(node)

```

```

true_body = if_node.insns
false_body = else_node.insns
if_stmt = self.gen_if_stmt(cond, true_body, false_body)
self.reduce_node(node, [if_stmt], [if_succ], True)
return True

```

- 要求 `if_node` 与 `else_node` 各自单后继，且二者汇合到同一 `join`；
- 生成标准 `if(cond){...} else {...}` 并保留汇合后继。

(E) 循环内控制语句规则

1. `try_reduce_if_continue(node, true_branch)`

```

def try_reduce_if_continue(self, node: CFGNode, true_branch: bool) -> bool:
    """尝试将给定的节点化简为if-continue结构，避免使用goto

```

具体来说：

识别如下CFG结构

```

node {stmt}
├─ if_node
│   └─ node (跳转到循环头)
└─ if_succ

```

将其化简为

```

node {stmt; if(cond) continue;}
└─ if_succ

```

:param node: 待化简的控制流图节点

:param true_branch: 尝试化简为if-continue结构的分支方向，为True时if语句的条件为`cond = get\_cond\_expr(node)`，

否则条件为`cond = cond.logical\_not()`

:return: 如果节点被化简为结构化语句，则返回True，否则返回False

"""

```

loop = self.get_loop_for(node)

```

```

if not loop:

```

```

    return False

```

```

succs: list[CFGNode] = self.successors(node)

```

```

if len(succs) != 2:

```

```

    return False

```

```

if_node = succs[0] if true_branch else succs[1]

```

```

if_succ = succs[1] if true_branch else succs[0]

```

```

if if_node == node or if_succ == node:

```

```

    return False

```

```

succs_if_node = self.successors(if_node)

```

```

if succs_if_node != [loop.header]:

```

```

    return False

```

```

cond = self.get_cond_expr(node)

```

```

if not true_branch:

```

```

        cond = cond.logical_not()

        continue_stmt = self.gen_continue_stmt(loop.header)
        body = list(if_node.insns)
        body.append(continue_stmt)
        if_stmt = self.gen_if_stmt(cond, body, None)
        self.reduce_node(node, [if_stmt], [if_succ], True)
        return True

```

- 前置: `node` 在某个循环内;
- 条件: `if_node` 的唯一后继是 `loop.header`;
- 生成 `if(cond){ <if_node语句>; continue; }`, 后继保留为另一分支 `if_succ`。

2. `try_reduce_if_break(node, true_branch)`

```

def try_reduce_if_break(self, node: CFGNode, true_branch: bool) -> bool:
    """尝试将给定的节点化简为if-break结构, 避免使用goto

```

具体来说::

```

    识别如下CFG结构
    node {stmt}
    |— if_node
    |   |— exit (跳转到内层循环出口)
    |   |— if_succ
    将其化简为
    node {stmt; if(cond) break;}
    |— if_succ

```

:param node: 待化简的控制流图节点

:param true_branch: 尝试化简为if-break结构的分支方向, 为True时if语句的条件为`cond = get\_cond\_expr(node)`,

否则条件为`cond = cond.logical\_not()`

:return: 如果节点被化简为结构化语句, 则返回True, 否则返回False
 """

```

    loop = self.get_loop_for(node)
    if not loop:
        return False

```

```

    succs: list[CFGNode] = self.successors(node)
    if len(succs) != 2:
        return False

```

```

    if_node = succs[0] if true_branch else succs[1]
    if_succ = succs[1] if true_branch else succs[0]

```

```

    if not self.is_loop_succ(if_node, loop):
        return False
    preds_if_node = self.predecessors(if_node)
    if len(preds_if_node) != 1 or preds_if_node[0] != node:
        return False
    succs_if_node = self.successors(if_node)

```

```

        if len(succs_if_node) != 1:
            return False
        # break分支中的节点应回到循环体中，表示“进入下一轮/继续循环”路径。
        if not self.get_loop_for(succs_if_node[0]) == loop and succs_if_node[0] not in
loop.body:
            return False

        cond = self.get_cond_expr(node)
        if not true_branch:
            cond = cond.logical_not()

        break_stmt = self.gen_break_stmt(loop.header)
        if_stmt = self.gen_if_stmt(cond, [break_stmt], None)
        self.reduce_node(node, [if_stmt], [if_succ], True)
        return True

```

- 前置：node 在循环内，且 break 分支目标满足循环出口约束；
- 通过前驱唯一性、后继唯一性和 loop 归属关系约束有效性；
- 生成 if(cond) break；并保留非 break 分支为后继。我在 if-break 上增加的结构约束（前驱唯一、后继唯一、循环归属检查）避免了误识别，能提升化简稳定性。

完整代码

```

"""
结构化代码生成器
"""

# pylint: disable=pointless-string-statement
# pylint: disable=unused-argument
# pylint: disable=fixme

from ..models.cfg import CFGNode
from ..models.expr import ConstExpr
from .structure_converter_mixin import StructureConverterMixin, ReduceRule

__all__ = ["StructureConverter"]

class StructureConverter(StructureConverterMixin):

    def reduce_cfg(self, node, by_rule, loop=None) -> bool:
        """递归化简控制流图，尝试将多个节点化简为一个节点。

        函数会使用广度优先搜索遍历控制流图中从给定节点可达的所有节点，并在尝试化简这些节点——将节点与其后继节点合并。

        :param node: 待化简的节点
        :param by_rule: 为True时使用针对性规则化简，否则使用通用(goto)规则
        :param loop: 当给定loop时，只化简循环内的节点，否则化简所有node可达的节点。
        :return: 如果节点被化简为结构化语句，则返回True
        """
        if by_rule:
            reduce_node_func = self.reduce_node_by_rule

```



```

else:
    reduce_node_func = self.reduce_node_by_goto

if loop:
    return self.bfs_traverse_loop(node, reduce_node_func, loop)
else:
    return self.bfs_traverse(node, reduce_node_func)

def reduce_node_by_goto(self, node: CFGNode) -> bool:
    """使用goto规则化简控制流图节点。

goto化简规则由`try_reduce_goto_`开头的函数实现。这些规则化简函数与
:meth:`reduce_node_by_rule`调用的规则化简函数类似；
区别在于这些规则化简函数只使用goto语句化简节点，并且一定能完成化简，并返回True。
因此通常情况下，方法一定会返回True，否则说明实现的goto规则不完整。

具体来说::

    识别如下结构
    node {stmt1}
      └─ succ {stmt2}
    将其化简为
    node {stmt1; goto stmt2;}

:param node: 待化简的控制流图节点
:return: 如果节点被化简，则返回True。注意：函数返回False说明函数实现不完整
"""
    if not self.successors(node):
        return False

    if self.try_reduce_goto_sequence(node):
        self.emit_reduce_event(ReduceRule.GOTO_SEQUENCE, node)
        return True
    if self.try_reduce_goto_branch(node):
        self.emit_reduce_event(ReduceRule.GOTO_BRANCH, node)
        return True
    if self.try_reduce_goto_switch(node):
        self.emit_reduce_event(ReduceRule.GOTO_SWITCH, node)
        return True

    self.log.warning(
        f"goto规则不全面无法化简节点: {node.address:#X}, "
        f"该节点有{len(self.successors(node))}个后继节点"
    )
    return False

def reduce_node_by_rule(self, node: CFGNode) -> bool:
    """检查给定的控制流图节点是否满足化简条件，如果可以则将其化简为结构化语句。

```

函数的实现逻辑是：首先检查节点是否为循环头，如果是则尝试化简循环；否则尝试使用其他规则化简节点。每个规则化简函数名以`try\_reduce\_`开头，例如`try\_reduce\_sequence`。

每个规则化简函数需要完成两方面操作：

1. 判断节点是否满足化简条件，如果不满足则返回False。

2. 如果满足则将节点化简为结构化语句，并返回True。

当任意节点被化简时，函数会调用`emit\_reduce\_event`方法记录化简事件，并返回True。

:param node: 待检查的控制流图节点

:return: 如果节点被化简为结构化语句，则返回True，否则返回False

"""

```
if self.is_loop_header(node) and not self.is_loop_header_under_analysis(node):
    self.enter_loop(node)
    self.completely_reduce_loop(node)
    self.leave_loop()
    return True
```

```
if self.try_reduce_sequence(node):
    self.emit_reduce_event(ReduceRule.SEQUENCE, node)
    return True
```

```
if self.try_reduce_while_true(node):
    self.emit_reduce_event(ReduceRule.WHILE_TRUE, node)
    return True
```

```
if self.try_reduce_do_while(node):
    self.emit_reduce_event(ReduceRule.DO_WHILE, node)
    return True
```

```
if self.try_reduce_if_else(node):
    self.emit_reduce_event(ReduceRule.IF_ELSE, node)
    return True
```

```
if self.try_reduce_if_continue(node, True) or self.try_reduce_if_continue(node,
False):
    self.emit_reduce_event(ReduceRule.IF_CONTINUE, node)
    return True
```

```
if self.try_reduce_if_break(node, True) or self.try_reduce_if_break(node, False):
    self.emit_reduce_event(ReduceRule.IF_BREAK, node)
    return True
```

```
if self.try_reduce_if(node, True) or self.try_reduce_if(node, False):
    self.emit_reduce_event(ReduceRule.IF, node)
    return True
```

```
return False
```

```
def completely_reduce_loop(self, node: CFGNode):
```

"""完全化简以节点为循环头的循环，将循环内的节点化简为结构化语句。

该方法首先尝试使用针对性规则化简循环；如果无法化简则使用goto规则将循环内的后继节点与节点合并，直到循环内只剩一个节点，从而可以将该节点转化为while(1)循环。

注意：如果循环没有被化简，大概率是通用(goto)规则实现不完整。

:param node: 循环头部节点

"""

```

loop = self.get_loop_for(node)

while not self.is_loop_reduced(loop) and self.reduce_cfg(node, by_rule=True):
    pass

if not self.is_loop_reduced(loop):
    self.reduce_cfg(node, by_rule=False, loop=loop)
    self.cfg.add_edge(node, node)
    if self.try_reduce_while_true(node):
        self.emit_reduce_event(ReduceRule.WHILE_TRUE, node)

```

以下是具体的控制流图结构识别和化简方法，其中有一部分方法未实现，请根据文档要求实现这些方法

```
def try_reduce_goto_sequence(self, node: CFGNode) -> bool:
```

"""使用goto规则化简顺序结构。对于给定的节点，如果只有一个后继节点，则在给定节点后面添加goto语句，并删除后继节点。

具体来说::

```

识别如下结构
node {stmt1}
└─ succ {stmt2}
将其化简为
node {stmt1; goto stmt2;}

```

:param node: 待化简的控制流图节点

:return: 如果节点被化简为结构化语句，则返回True，否则返回False

"""

```

succs: list[CFGNode] = self.successors(node)

```

若后继数量不为1，则不符合化简条件

```

if len(succs) != 1:
    return False

```

```

goto_node = succs[0]

```

排除自循环的情况

```

if goto_node == node:
    return False

```

若后继节点仅由当前节点到达，则优先直接拼接语句，减少残留goto目标块。

```

preds = self.predecessors(goto_node)

```

```

if len(preds) == 1 and preds[0] == node:

```

```

    self.reduce_node(node, goto_node.insns, self.successors(goto_node), True)
    return True

```

```

goto_stmt = self.gen_goto_stmt(goto_node)

```

```

self.reduce_node(node, [goto_stmt], new_succs=[], extend=True)

```

```

return True

```

```
def try_reduce_goto_branch(self, node: CFGNode) -> bool:
```

"""使用goto规则化简if-else结构。对于给定的节点，如果有两个后继节点，则在节点后面添加if-goto-else-goto语句，并删除后继节点。

具体来说::

识别如下结构

```
node {stmt1}
├─ succ1 {stmt2}
└─ succ2 {stmt3}
```

将其化简为

```
node {stmt1; if(cond) goto stmt2; else goto stmt3;}
```

:param node: 待化简的控制流图节点

:return: 如果节点被化简为结构化语句, 则返回True, 否则返回False

"""

```
succs: list[CFGNode] = self.successors(node)
```

```
# 若后继数量不为2, 则不符合化简条件
```

```
if len(succs) != 2:
```

```
    return False
```

```
cond = self.get_cond_expr(node)
```

```
succs1_node = succs[0]
```

```
succs2_node = succs[1]
```

```
goto_stmt1 = self.gen_goto_stmt(succs1_node)
```

```
goto_stmt2 = self.gen_goto_stmt(succs2_node)
```

```
if_stmt = self.gen_if_stmt(cond, [goto_stmt1], [goto_stmt2])
```

```
self.reduce_node(node, [if_stmt], new_succs=[], extend=True)
```

```
return True
```

```
def try_reduce_goto_switch(self, node: CFGNode) -> bool:
```

```
    """使用goto规则化简switch结构。不要求实现"""
```

```
    return False
```

```
def try_reduce_while_true(self, node: CFGNode) -> bool:
```

```
    """尝试将给定的节点化简为while(1)语句, 避免使用goto
```

具体来说::

识别如下自循环结构

```
node {stmt}
└─ node (自循环)
```

将其化简为

```
node {while(1) {stmt}}
```

:param node: 待化简的控制流图节点

:return: 如果节点被化简为结构化语句, 则返回True, 否则返回False

"""

```
succs: list[CFGNode] = self.successors(node)
```

```
# 判断是否为自循环
```

```
if len(succs) != 1 or succs[0] != node:
```

```
    return False
```

```
while_stmt = self.gen_while_stmt(ConstExpr(1), node.insns)
```

```
self.reduce_node(node, [while_stmt], new_succs=[], extend=False)
```

```
return True
```

```
def try_reduce_do_while(self, node: CFGNode) -> bool:
```

```
    """尝试将给定的节点化简为do-while语句, 避免使用goto
```

具体来说::

识别如下结构

```
node {stmt}  
├─ node (自循环)  
└─ succ (循环后继节点)
```

将其化简为

```
node {do {stmt} while (1)}  
└─ succ
```

:param node: 待化简的控制流图节点

:return: 如果节点被化简为结构化语句, 则返回True, 否则返回False

"""

```
succs: list[CFGNode] = self.successors(node)
```

```
# 判断是否为do_while循环
```

```
if len(succs) != 2:
```

```
    return False
```

```
if node not in succs:
```

```
    return False
```

```
# 确定循环后继节点
```

```
loop_succ = succs[1] if succs[0] == node else succs[0]
```

```
cond = self.get_cond_expr(node)
```

```
# get_cond_expr(node) 表示 True 分支 (succs[0]) 条件,
```

```
# 若自循环在 False 分支上, 需要取反。
```

```
if succs[1] == node:
```

```
    cond = cond.logical_not()
```

```
do_while_stmt = self.gen_do_while_stmt(cond, node.insns)
```

```
self.reduce_node(node, [do_while_stmt], new_succs=[loop_succ], extend=False)
```

```
return True
```

```
def try_reduce_sequence(self, node: CFGNode) -> bool:
```

```
    """尝试将简单顺序结构化简为单个节点, 避免使用goto
```

具体来说::

识别如下结构

```
node {stmt1}  
└─ succ {stmt2} (succ != node, 即不是自循环)  
...  
└─ succ_succ_N (N >= 0)
```

将其化简为

```
node {stmt1; stmt2}  
...  
└─ succ_succ_N (N >= 0)
```

:param node: 待化简的控制流图节点

:return: 如果节点被化简为结构化语句, 则返回True, 否则返回False

"""

```
succs: list[CFGNode] = self.successors(node)
```

```
# 判断是否为顺序结构
```

```
if len(succs) != 1:
```

```

        return False

    succ_node = succs[0]
    # 判断是否为自循环
    if succ_node == node:
        return False

    # 判断是否有其他前驱节点
    preds = self.predecessors(succ_node)
    if len(preds) != 1 or preds[0] != node:
        return False

    self.reduce_node(node, succ_node.insns, self.successors(succ_node), True)
    return True

```

```
def try_reduce_if(self, node: CFGNode, true_branch: bool) -> bool:
```

"""尝试将给定的节点化简为if结构，避免使用goto。

当branch为True时，将node的第一个后继视为if_node，否则将另一个后继视为if_node

具体来说：

识别如下结构：

```

node {stmt}
├─ if_node {stmt1}
│   └─ if_succ
└─ if_succ

```

将其化简为

```

node {stmt; if(cond) {stmt1}}
└─ if_succ

```

:param node: 待化简的控制流图节点

:param true_branch: 尝试化简为if结构的分支方向，为True时if语句的条件为`cond = get\_cond\_expr(node)`，

否则条件为`cond = cond.logical\_not()`

:return: 如果节点被化简为结构化语句，则返回True，否则返回False

"""

```
    succs: list[CFGNode] = self.successors(node)
```

```
    if len(succs) != 2:
```

```
        return False
```

```
    if_node = succs[0] if true_branch else succs[1]
```

```
    if_succ = succs[1] if true_branch else succs[0]
```

```
    if if_node == node or if_succ == node:
```

```
        return False
```

判断if_node的后继节点是否为if_succ

```
    succs_if_node = self.successors(if_node)
```

```
    if succs_if_node != [if_succ]:
```

```
        return False
```

```
    cond = self.get_cond_expr(node)
```

```
    if not true_branch:
```

```
        cond = cond.logical_not()
```

```

if_stmt = self.gen_if_stmt(cond, if_node.insns, None)
self.reduce_node(node, [if_stmt], [if_succ], True)
return True

```

```

def try_reduce_if_continue(self, node: CFGNode, true_branch: bool) -> bool:
    """尝试将给定的节点化简为if-continue结构，避免使用goto

```

具体来说::

```

    识别如下CFG结构
    node {stmt}
    |— if_node
    |   └─ node (跳转到循环头)
    └─ if_succ
    将其化简为
    node {stmt; if(cond) continue;}
    └─ if_succ

```

:param node: 待化简的控制流图节点

:param true_branch: 尝试化简为if-continue结构的分支方向，为True时if语句的条件为`cond = get\_cond\_expr(node)`，

否则条件为`cond = cond.logical\_not()`

:return: 如果节点被化简为结构化语句，则返回True，否则返回False
 """

```

loop = self.get_loop_for(node)

```

```

if not loop:
    return False

```

```

succs: list[CFGNode] = self.successors(node)
if len(succs) != 2:
    return False

```

```

if_node = succs[0] if true_branch else succs[1]
if_succ = succs[1] if true_branch else succs[0]
if if_node == node or if_succ == node:
    return False

```

```

succs_if_node = self.successors(if_node)
if succs_if_node != [loop.header]:
    return False

```

```

cond = self.get_cond_expr(node)
if not true_branch:
    cond = cond.logical_not()

```

```

continue_stmt = self.gen_continue_stmt(loop.header)
body = list(if_node.insns)
body.append(continue_stmt)
if_stmt = self.gen_if_stmt(cond, body, None)
self.reduce_node(node, [if_stmt], [if_succ], True)
return True

```

```
def try_reduce_if_break(self, node: CFGNode, true_branch: bool) -> bool:
    """尝试将给定的节点化简为if-break结构，避免使用goto
```

具体来说::

识别如下CFG结构

```
node {stmt}
├─ if_node
│   └─ exit (跳转到内层循环出口)
└─ if_succ
```

将其化简为

```
node {stmt; if(cond) break;}
└─ if_succ
```

:param node: 待化简的控制流图节点

:param true_branch: 尝试化简为if-break结构的分支方向，为True时if语句的条件为`cond = get\_cond\_expr(node)`，

否则条件为`cond = cond.logical\_not()`

:return: 如果节点被化简为结构化语句，则返回True，否则返回False

"""

```
loop = self.get_loop_for(node)
```

```
if not loop:
```

```
    return False
```

```
succs: list[CFGNode] = self.successors(node)
```

```
if len(succs) != 2:
```

```
    return False
```

```
if_node = succs[0] if true_branch else succs[1]
```

```
if_succ = succs[1] if true_branch else succs[0]
```

```
if not self.is_loop_succ(if_node, loop):
```

```
    return False
```

```
preds_if_node = self.predecessors(if_node)
```

```
if len(preds_if_node) != 1 or preds_if_node[0] != node:
```

```
    return False
```

```
succs_if_node = self.successors(if_node)
```

```
if len(succs_if_node) != 1:
```

```
    return False
```

```
# break分支中的节点应回到循环体中，表示“进入下一轮/继续循环”路径。
```

```
if not self.get_loop_for(succs_if_node[0]) == loop and succs_if_node[0] not in
```

```
loop.body:
```

```
    return False
```

```
cond = self.get_cond_expr(node)
```

```
if not true_branch:
```

```
    cond = cond.logical_not()
```

```
break_stmt = self.gen_break_stmt(loop.header)
```

```
if_stmt = self.gen_if_stmt(cond, [break_stmt], None)
```

```
self.reduce_node(node, [if_stmt], [if_succ], True)
```

```
return True
```



```
def try_reduce_if_else(self, node: CFGNode) -> bool:
```

```
    """尝试将给定的节点化简为if-else结构，避免使用goto
```

```
    具体来说::
```

```
        识别如下CFG结构
```

```
        node {stmt}
```

```
        |— if_node {stmt1}
```

```
        |   └─ if_succ
```

```
        └─ else_node {stmt2}
```

```
            └─ else_succ (if_succ == else_succ)
```

```
    将其化简为
```

```
    node {stmt; if(cond) {stmt1} else {stmt2};}
```

```
    └─ if_succ
```

```
:param node: 待化简的控制流图节点
```

```
:return: 如果节点被化简为结构化语句，则返回True，否则返回False
```

```
"""
```

```
    succs = self.successors(node)
```

```
    if len(succs) != 2:
```

```
        return False
```

```
    if_node = succs[0]
```

```
    else_node = succs[1]
```

```
    if if_node == node or else_node == node:
```

```
        return False
```

```
    if if_node == else_node:
```

```
        return False
```

```
    if_succs = self.successors(if_node)
```

```
    if len(if_succs) != 1:
```

```
        return False
```

```
    else_succs = self.successors(else_node)
```

```
    if len(else_succs) != 1:
```

```
        return False
```

```
    if_succ = if_succs[0]
```

```
    else_succ = else_succs[0]
```

```
    if if_succ != else_succ:
```

```
        return False
```

```
    cond = self.get_cond_expr(node)
```

```
    true_body = if_node.insns
```

```
    false_body = else_node.insns
```

```
    if_stmt = self.gen_if_stmt(cond, true_body, false_body)
```

```
    self.reduce_node(node, [if_stmt], [if_succ], True)
```

```
    return True
```

额外的优化

我修改了 `poredec/backend/structure_converter_mixin.py`。我主要改了 `merge_goto_targets` 相关的内联合并策略，抑制 goto 目标的重复展开与错误内联。

```
def merge_goto_targets(self):
    """尝试将goto语句替换为目标代码块，如果不能替换，则将目标代码块添加到CFG入口节点"""
    cfg = self.cfg

    goto_stmt_target_map = {}
    for goto_target, goto_stmts in self._goto_targets.items():
        # 单语句循环目标(while/do-while)不能内联到goto位置，否则会把“跳回循环”展开成整段循环体。
        is_loop_stmt_target = (
            len(goto_target.insns) == 1
            and isinstance(goto_target.insns[0], (while_stmt, DowhileStmt))
        )
        is_exit_target = not self.successors(goto_target)
        if is_loop_stmt_target:
            continue
        if is_exit_target and len(goto_stmts) != 1:
            continue
        for goto_stmt in goto_stmts:
            goto_stmt_target_map[goto_stmt] = goto_target

    graph = nx.DiGraph()
    queue = Queue()
    for node in list(cfg.nodes):
        queue.put((node, node.insns))
    # 遍历控制流图，找到所有可能可以替换的goto语句的位置
    while not queue.empty():
        node, insns = queue.get()
        for idx, stmt in enumerate(insns):
            if isinstance(stmt, GotoStmt):
                if stmt not in goto_stmt_target_map:
                    continue
                goto_target = goto_stmt_target_map.get(stmt)
                if not graph.has_edge(node, goto_target):
                    graph.add_edge(node, goto_target, tasks=[])
                    graph.get_edge_data(node, goto_target)["tasks"].append((insns, idx))
            elif isinstance(stmt, IfStmt):
                queue.put((node, stmt.true_body))
                if stmt.false_body:
                    queue.put((node, stmt.false_body))
            elif isinstance(stmt, while_stmt) or isinstance(stmt, DowhileStmt):
                queue.put((node, stmt.body))

    # 删除循环goto
    no_merge_nodes = set()
    for cycle in nx.simple_cycles(graph):
        no_merge_nodes.update(cycle)
    graph.remove_nodes_from(no_merge_nodes)

    remaining_goto_targets = set(self._goto_targets.keys())
```

```

for node in reversed(list(nx.topological_sort(graph))):
    goto_targets = list(graph.successors(node))
    if not goto_targets:
        continue
    for goto_target in goto_targets:
        tasks = graph.get_edge_data(node, goto_target)["tasks"]
        # 同一语句列表内按逆序替换，避免前面替换后影响后面索引。
        by_list_id: dict[int, list[tuple[list[Statement], int]]] = {}
        for insns, idx in tasks:
            by_list_id.setdefault(id(insns), []).append((insns, idx))
        for grouped_tasks in by_list_id.values():
            uniq_by_idx: dict[int, list[Statement]] = {}
            for insns, idx in grouped_tasks:
                uniq_by_idx.setdefault(idx, insns)
            # 同一列表对同一目标通常只需要替换一次；多次替换易造成同块重复展开。
            idx = max(uniq_by_idx.keys())
            insns = uniq_by_idx[idx]
            self.log.debug(
                "替换goto语句: %#X[pos:%d,size:%d] -> %#X[size:%d]",
                node.address,
                idx,
                len(insns),
                goto_target.address,
                len(goto_target.insns),
            )
            target_label = f"LAB_{goto_target.address:X}"
            if any(getattr(stmt, "label", None) == target_label for stmt in insns):
                continue
            insns[idx : idx + 1] = goto_target.insns
            remaining_goto_targets.discard(goto_target)

```

核心有 4 类：

- 新增“循环目标不内联”保护
 - 对目标块是单条 `whileStmt/DowhileStmt` 的情况，直接 `continue` 不做内联。
 - 目的：避免把“跳回循环头”的 `goto` 展开成整段循环体，导致语义膨胀。
- 调整退出块（exit target）合并条件
 - 新增 `is_exit_target = not self.successors(goto_target)`。
 - 现在只有“出口目标且只有一个 `goto` 指向”才会内联；多个引用时不内联。
 - 目的：避免出口块被多处重复展开。
- 重写 `goto` 替换逻辑，避免索引错乱和重复展开
 - 原来用 `base_idx` 顺序替换；你改成：
 - 按 `insns` 列表分组（`by_list_id`）
 - 同一列表内按索引去重
 - 只对该目标取一次替换（取最大 `idx`）
 - 并加了标签存在检查：若当前列表已含 `LAB_XXX`，跳过替换。
 - 目的：防止同一块内多次展开同一个目标，避免替换时前后索引互相影响。

- 新增递归 `has_label()`，防止尾部重复追加目标块
 - 递归检查 `IfStmt`、`WhileStmt`、`DowhileStmt` 内部是否已有目标标签。
 - 在把剩余 `goto_target` 追加到 `cfg.entry` 前，先检查是否已存在标签，存在就不追加。
 - 目的：防止目标块被“内联过一次又在入口末尾再追加一次”。

2.3 结果分析

1) SEQUENCE：sequence_1052C

sequence_1052C.simplified.c	decompiled_simplified.c
1 // 首指令地址: 0X1052C	1 // 首指令地址: 0X1052C
2 // 末指令地址: 0X105A0	2 // 末指令地址: 0X105A0
3 if (!ZR) {	3 if (!ZR) {
4 // 首指令地址: 0X105A4	4 // 首指令地址: 0X105A4
5 // 末指令地址: 0X105A4	5 // 末指令地址: 0X105A4
6 }	6 }
7 // 首指令地址: 0X105A8	7 // 首指令地址: 0X105A8
8 // 末指令地址: 0X105B0	8 // 末指令地址: 0X105B0
9	9

- 现象：多个线性基本块被逐步合并为单节点语句序列；
- 结果：输出简化代码中无多余跳转，语句顺序与执行顺序一致；
- 正确性依据：每次合并都满足“后继唯一且前驱唯一”，不会跨越分支边界，语义保持不变。

2) WHILE_TRUE：loop_while_true_10420

loop_while_true_10420.simplified.c	decompiled_simplified.c
1 // 首指令地址: 0X10420	1 // 首指令地址: 0X10420
2 // 末指令地址: 0X10438	2 // 末指令地址: 0X10438
3 while (1) {	3 while (1) {
4 // 首指令地址: 0X1043C	4 // 首指令地址: 0X1043C
5 // 末指令地址: 0X10444	5 // 末指令地址: 0X10444
6 }	6 }
7	7

- 现象：自环节点被识别为无限循环头；
- 结果：生成 `while (1) { ... }` 结构；
- 正确性依据：该规则仅在单自环成立时触发，严格等价于原 CFG 的“每轮必回头”语义。

3) DO_WHILE: loop_while_10420

loop_while_10420.simplified.c	decompiled_simplified.c
benchmarks > references > loop_while_10420.simplified.c	outputs > loop_while_10420 > decompiled_simplified.c
<pre>1 // 首指令地址: 0X10420 2 // 末指令地址: 0X10444 3 while (1) { 4 // 首指令地址: 0X10470 5 // 末指令地址: 0X1047C 6 if (unique_2280_1) { 7 // 首指令地址: 0X10448 8 // 末指令地址: 0X1046C 9 continue 10 } 11 // 首指令地址: 0X10480 12 // 末指令地址: 0X1048C 13 } 14</pre>	<pre>1 // 首指令地址: 0X10420 2 // 末指令地址: 0X10444 3 while (1) { 4 // 首指令地址: 0X10470 5 // 末指令地址: 0X1047C 6 if (unique_2280_1) { 7 // 首指令地址: 0X10448 8 // 末指令地址: 0X1046C 9 continue 10 } 11 // 首指令地址: 0X10480 12 // 末指令地址: 0X1048C 13 } 14</pre>

- 现象：节点有自环和退出两条边，匹配“先执行后判定”模式；
- 结果：生成 `do { ... } while(cond)` 结构并保留循环后继；
- 正确性依据：对分支方向做了条件翻转处理，保证循环继续条件与真实跳转方向一致。

4) IF: branch_if_10420

branch_if_10420.simplified.c	decompiled_simplified.c
benchmarks > references > branch_if_10420.simplified.c	outputs > branch_if_10420 > decompiled_simplified.c
<pre>1 // 首指令地址: 0X10420 2 // 末指令地址: 0X10444 3 if (ZR) { 4 // 首指令地址: 0X10458 5 // 末指令地址: 0X10458 6 } else { 7 // 首指令地址: 0X10448 8 // 末指令地址: 0X10454 9 } 10 // 首指令地址: 0X1045C 11 // 末指令地址: 0X10464 12</pre>	<pre>1 // 首指令地址: 0X10420 2 // 末指令地址: 0X10444 3 if (ZR) { 4 // 首指令地址: 0X10458 5 // 末指令地址: 0X10458 6 } else { 7 // 首指令地址: 0X10448 8 // 末指令地址: 0X10454 9 } 10 // 首指令地址: 0X1045C 11 // 末指令地址: 0X10464 12</pre>

- 现象：一条分支进入单节点 if 体，另一条直接汇合；
- 结果：生成 `if (cond) { ... }`；
- 正确性依据：`if_node` 的唯一后继是汇合点，满足典型单分支结构判定条件。

5) IF_ELSE: `branch_if_else_10420`

branch_if_else_10420.simplified.c	decompiled_simplified.c
1 // 首指令地址: 0X10420	1 // 首指令地址: 0X10420
2 // 末指令地址: 0X10444	2 // 末指令地址: 0X10444
3 if (ZR) {	3 if (ZR) {
4 // 首指令地址: 0X10458	4 // 首指令地址: 0X10458
5 // 末指令地址: 0X10468	5 // 末指令地址: 0X10468
6 } else {	6 } else {
7 // 首指令地址: 0X10448	7 // 首指令地址: 0X10448
8 // 末指令地址: 0X10454	8 // 末指令地址: 0X10454
9 }	9 }
10 // 首指令地址: 0X1046C	10 // 首指令地址: 0X1046C
11 // 末指令地址: 0X10474	11 // 末指令地址: 0X10474
12	12

- 现象：真假两分支各有独立语句体并在同一点汇合；
- 结果：生成标准 `if-else` 结构；
- 正确性依据：两分支都要求单后继且同汇合点，能保证结构化后控制流等价。

6) IF_CONTINUE: `loop_while_continue_10450`

loop_while_continue_10450.simplified.c	decompiled_simplified.c
1 // 首指令地址: 0X10450	1 // 首指令地址: 0X10450
2 // 末指令地址: 0X10474	2 // 末指令地址: 0X10474
3 while (1) {	3 while (1) {
4 // 首指令地址: 0X104A4	4 // 首指令地址: 0X104A4
5 // 末指令地址: 0X104AC	5 // 末指令地址: 0X104AC
6 if (unique_2600_1) {	6 if (unique_2600_1) {
7 // 首指令地址: 0X10478	7 // 首指令地址: 0X10478
8 // 末指令地址: 0X1048C	8 // 末指令地址: 0X1048C
9 if (unique_2600_1) {	9 if (unique_2600_1) {
10 // 首指令地址: 0X104A0	10 // 首指令地址: 0X104A0
11 // 末指令地址: 0X104A0	11 // 末指令地址: 0X104A0
12 } else {	12 } else {
13 // 首指令地址: 0X10490	13 // 首指令地址: 0X10490
14 // 末指令地址: 0X1049C	14 // 末指令地址: 0X1049C
15 }	15 }
16 continue	16 continue
17 }	17 }
18 // 首指令地址: 0X104B0	18 // 首指令地址: 0X104B0
19 // 末指令地址: 0X104C4	19 // 末指令地址: 0X104C4
20 }	20 }
21	21

- 现象：循环体中某分支回到循环头，另一分支继续向后；
- 结果：生成 `if (cond) continue;`（可带分支体语句）；
- 正确性依据：规则要求该分支唯一后继必须是 `loop.header`，确保 `continue` 语义精确。

7) IF_BREAK: httpd_209AC

httpd_209AC.simplified.c ×

benchmarks > references > httpd_209AC.simplified.c

```
1 // 首指令地址: 0X209AC
2 // 末指令地址: 0X209C4
3 while (1) {
4     // 首指令地址: 0X20A94
5     // 末指令地址: 0X20A9C
6     if (unique_1a80_1) {
7         // 首指令地址: 0X209C8
8         // 末指令地址: 0X209CC
9         do {
10             // 首指令地址: 0X209D0
11             // 末指令地址: 0X209F4
12         } while (unique_1a80_1);
13         // 首指令地址: 0X209F8
14         // 末指令地址: 0X20A24
15         if (ZR) {
16             // 首指令地址: 0X20A34
17             // 末指令地址: 0X20A48
18         } else {
19             // 首指令地址: 0X20A28
20             // 末指令地址: 0X20A2C
21             if (unique_1a80_1) {
22                 // 首指令地址: 0X20A68
23                 // 末指令地址: 0X20A88
24             } else {
25                 // 首指令地址: 0X20A30
26                 // 末指令地址: 0X20A64
27             }
28             // 首指令地址: 0X20A8C
29             // 末指令地址: 0X20A8C
30         }
31         // 首指令地址: 0X20A90
32         // 末指令地址: 0X20A90
33         continue
34     }
35     // 首指令地址: 0X20AA0
36     // 末指令地址: 0X20AAC
37 }
38
```

decompiled_simplified.c ×

outputs > httpd_209AC > decompiled_simplified.c

```
1 // 首指令地址: 0X209AC
2 // 末指令地址: 0X209C4
3 while (1) {
4     // 首指令地址: 0X20A94
5     // 末指令地址: 0X20A9C
6     if (unique_1a80_1) {
7         // 首指令地址: 0X209C8
8         // 末指令地址: 0X209CC
9         do {
10             // 首指令地址: 0X209D0
11             // 末指令地址: 0X209F4
12         } while (unique_1a80_1);
13         // 首指令地址: 0X209F8
14         // 末指令地址: 0X20A24
15         if (ZR) {
16             // 首指令地址: 0X20A34
17             // 末指令地址: 0X20A48
18         } else {
19             // 首指令地址: 0X20A28
20             // 末指令地址: 0X20A2C
21             if (unique_1a80_1) {
22                 // 首指令地址: 0X20A68
23                 // 末指令地址: 0X20A88
24             } else {
25                 // 首指令地址: 0X20A30
26                 // 末指令地址: 0X20A64
27             }
28             // 首指令地址: 0X20A8C
29             // 末指令地址: 0X20A8C
30         }
31         // 首指令地址: 0X20A90
32         // 末指令地址: 0X20A90
33         continue
34     }
35     // 首指令地址: 0X20AA0
36     // 末指令地址: 0X20AAC
37 }
38
```

- 现象：循环内条件分支触发退出路径；
- 结果：生成 `if (cond) break;` 并保留循环内继续执行路径；
- 正确性依据：在规则中加入了循环归属与边唯一性约束，避免将普通跳转误判为 `break`。

8) GOTO_SEQUENCE / GOTO_BRANCH: httpd_20814

httpd_20814.simplified.c

benchmarks > references > httpd_20814.simplified.c

1

2

3

4

5

6

7

8

9

// 首指令地址: 0X20814

// 末指令地址: 0X2084C

if (!ZR) {

// 首指令地址: 0X20850

// 末指令地址: 0X2085C

}

// 首指令地址: 0X20860

// 末指令地址: 0X20864

decompiled_simplified.c

outputs > httpd_20814 > decompiled_simplified.c

1

2

3

4

5

6

7

8

9

// 首指令地址: 0X20814

// 末指令地址: 0X2084C

if (!ZR) {

// 首指令地址: 0X20850

// 末指令地址: 0X2085C

}

// 首指令地址: 0X20860

// 末指令地址: 0X20864

- 现象：复杂局部结构在针对性规则不足以覆盖时，自动退化为 goto 化简；
- 结果：CFG 仍可继续收缩，最终完成全图化简；
- 正确性依据：goto 规则保持控制转移显式可追踪，语义保守且完整。

总结

通过本项目，我对“反编译前端构图 + 后端结构恢复”的完整链路建立了系统理解：

1. **CFG 是结构化恢复的前提**：基本块边界与边语义一旦偏差，后续规则识别会级联失真。
2. **规则设计本质是图模式匹配**：每条化简规则都需要“触发条件 + 语义保持 + 图收缩”三位一体。
3. **工程细节决定可用性**：例如可达性过滤、跳板块合并、条件取反时机、循环上下文校验，这些细节直接决定输出质量。
4. **通用规则与针对性规则互补**：针对性规则提升可读性，goto 规则保证完备性；两者配合可实现“尽量结构化且必定可收敛”。